

Aula 06 – Introdução às string e Arquivos em Python 3

Parte 1 – Revisão Teórica

Introdução às string

Uma string é qualquer sequência de caracteres. Caractere é a menor unidade de todo o texto.

Os caracteres são classificados em 3 grupos:

- a) **numérico,**
- b) **letras do alfabeto e**
- c) **caracteres especiais.**

É comum nos referirmos ao conjunto de caracteres numéricos e as letras com o termo alfanumérico.

Na programação, é comum a manipulação de cadeias de caracteres, isso porque, a maior parte da informação que o ser humano trabalha, está no formato de texto. Assim, processamos caracteres a todo instante e rapidamente descobrimos, a grande necessidade em tratar e processar textos. E, rapidamente aprendemos que existem centenas de maneiras para tratarmos e reconhecermos trechos de texto.

- 1) "Texto entre aspas Simples"
- 2) "Texto entre aspas Dupla"
- 3) "'String com 3 aspas Simples'"
- 4) "'''String com 3 aspas Duplas'''"

Listagem A: Formas de definição de String em Python

```
textoApasSimples = "Texto entre aspas Simples"
textoAspasDuplas = "Texto entre aspas Dupla"
texto3AspasSimples = "'String com 3 aspas Simples,'
texto2,
texto3,
texto4'"

texto3AspasDuplas = "'''String com 3 aspas Duplas,
textoA,
textoB,
textoC'''"

print(textoApasSimples)
print(textoAspasDuplas)
print(texto3AspasSimples)
print(texto3AspasDuplas )
```

Listagem 1: Exemplo60.py

Por exemplo, toda linguagem de programação precisa ser interpretada e, para isso, deve haver o reconhecimento de diversos nomes, diversas partes. Cada parte, ou melhor, cada segmento de texto reconhecido possua significado diferente, algumas vezes serão expressões, noutras, apenas sinais.

Por essas razões, o entendimento e o domínio da manipulação de caracteres e de conjuntos de caracteres é tão essencial.

Para o Python, **todo texto** ou **caractere** é um objeto do **tipo String (str)**. Portanto, se precisarmos utilizar determinado texto como sendo um outro tipo de informação, é nossa responsabilidade como programador, transformar, ou melhor, converter do **tipo str** para o novo tipo.

Vamos supor que desejamos que determinada **String** seja reconhecido pela linguagem como sendo um número, então, precisamos fazer a conversão e essa pode ser feita da seguinte forma:

```
texto1 = "10"
texto2 = "20.2"

num = int(texto1)
valor = float(texto2)

print("Valor de variável num = ", num)
print("Valor da variável valor = ", valor)
```

Listagem 2

Já estudamos, que a utilização do sinal de adição + quando utilizado com 2 objetos do tipo String, ao invés de realizar a operação aritmética, fará a concatenação ou junção do primeiro com a segunda parte. Então, quando estivermos lidando com números, desejaremos obter, com o sinal de adição, soma entre 2 valores numéricos. Por isso, sempre precisaremos converter de str para o tipo que desejamos e para que nossas conversões funcionem, muitas vezes, precisaremos saber, se os caracteres são válidos.

O conceito de caractere e string

Caractere é a unidade mínima de toda String. Um caractere pode ser numérico, letras do alfabeto ou um caractere especial.

É importante observar, que no Python, diferentemente de outras linguagens, nós não temos o tipo Char, ou seja, o tipo caractere.

Para o Python, **tudo são Strings**, indiferente se a mesma contiver 1 ou "n" caracteres.

No trecho de código a seguir, utilizamos a função **type()** para saber qual o tipo de alguns objetos, veja os resultados e abra o seu IDLE e faça os testes você também.

```
texto1 = "10"
texto2 = "20.2"

num = int(texto1)
valor = float(texto2)
print()
print("Valor de variável texto1 = " + texto1)
print("Tipo da variavel texto1 = ", type(texto1))
print()
print("Valor de variável num = ", num)
print("Tipo da variavel num = ", type(num))
print()
print("Valor da variável valor = ", valor)
print("Tipo da variavel valor = ", type(valor))
```

Listagem 3

A seguir, implementamos um código simples onde um texto é vinculado a uma referência. Em seguida, a referência e o valor vinculado são impresso na saída padrão. Observe como utilizamos variáveis que contém caracteres:

```
s = "Lista de Caracteres em Python"
print(s)
```

Listagem 4

Terminologias

Quando manipulamos cadeias de caracteres, isto é, **string**, utilizamos algumas terminologias específicas que são, de fato, o nome da ação que será realizada.

Termo	Descrição
Concatenar	✓ Concatenar é a junção de 2 cadeias de caracteres e que dá origem a uma nova string que é formada pela junção das 2 partes. ✓ A concatenação, pode ser a direita ou então a esquerda de outra string. ✓ Então por exemplo, a listagem 4, temos um exemplo em que fazemos alguns tipos de concatenações.
interpoliar	✓ Interpoliar é a inserção de um trecho de texto dentro de outro. ✓ A interpolação, ocorre de várias formas, e, algumas linguagens e bibliotecas, proporcionam maneiras bastante interessantes e diferentes para interpolarmos valores. ✓ O Python fornece um conjunto de ferramentas bastante poderosas para esse fim.

Exemplos

```
#concatenação de 2 trechos de texto
texto1 = "123"
texto2 = "456"
concatenacao = texto1 + texto2
print("Resultado da variavel concatenação = " +concatenacao) # Saida = '123456'

#declaração da variável `a` que foi associado ao texto `bra`
a = "bra"

#declaração da variável `b` que foi associado ao texto `sil`
b = "sil"

#ao escrevemos no prompt a + b temos que é impresso a CONCATENAÇÃO
print("concatenação de a + b = " + (a + b)) # Saida = 'brasil'

#agora, concatenamos e o resultado associamos a variável `c`
c = a + b
print("Resutado da variavel c = " + c) #Saida = 'brasil'

#agora, podemos dizer que o trecho de texto `bra`
# foi adicionado a esquerda da variável `b`
print("concatenação de b + a = " + (b + a)) #Saida ='silbra'
```

Listagem 5: Exemplo de concatenar

```
sexo = "masculino"
nome = "Henrique"

interpoliar = "Sexo é igual a %s e o nome é igual %s"%(sexo, nome)
print("Saida 1 da interpolação = " +interpoliar)

#agora, vamos colocar uma String entre chaves dentro do texto
```

```
#em seguida, com a função format(), vamos associar
#os nomes definidos entre chaves a texto passando-os
#como argumentos de função

interpolador = "Sexo é igual a {SEXO} e o nome é igual {NOME} ".format(SEXO=sexo,NOME=nome)
print("Saída 2 da interpolação = " +interpolador)

#Saída = 'Sexo é igual a masculino e o nome é igual Henrique '
```

Listagem 6: Exemplo de interpolar

As string SÃO LISTAS IMUTÁVEIS

Para o Python, toda **string** é uma **lista imutável**, semelhante a uma **Tupla**, porém, com funcionalidade bastante específicas.

A manipulação de **string**, **tuplas** e **listas** é bastante semelhante, e, a definição em Python de **Tupla** e **String** é quase que idêntico.

Toda **String** após definida **NÃO pode ser alterada**, ou seja, você não pode alterar um caractere dentro de uma **String**.

Então, ao fazermos alguma alteração numa determinada parte de uma **String**, o que de fato ocorrerá é a definição de uma **nova String**.

Há várias razões pelas quais as **Strings** não podem ser alteradas, algumas delas são que, internamente, quando uma **string** é definida, uma série de otimizações são feitas, de tal forma, que é mais rápido destruir a instância existente e criar uma nova, do que reaproveitar as otimizações existentes.

Por isso, por exemplo, a concatenação, interpolação ou qualquer manipulação de **string**, resulta, invariavelmente, numa nova instância que é criada e que possui as modificações.

O fatiamento de string

No entanto, estudar o fatiamento com **string** pode ser mais fácil e intuitivo, até porque, trabalharemos diretamente com caracteres.

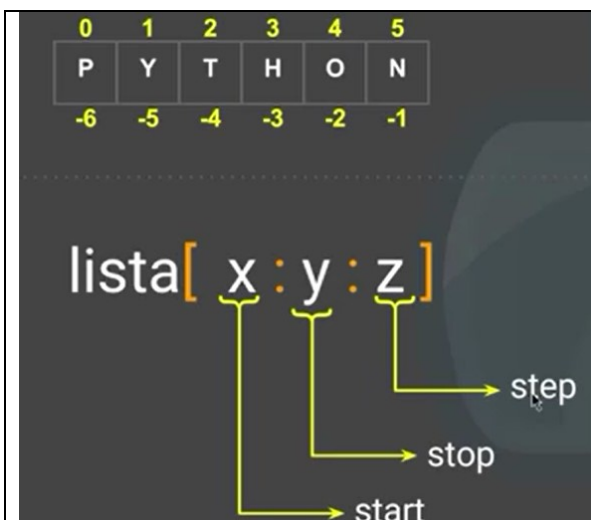


Figura 1:

O sistema de índices também é igual ao sistema de índices das Listas, logo, o primeiro elemento possui índice igual a **0** e o último, índice igual a **-1**.

A listagem 7 mostra um simples exemplo de como fatiar uma string.

```
texto = "Toda string em Python é imutável"

print("Frase completa = " + texto)

#retornando o primeiro caractere
print("Retorna o primeiro caractere da variavel texto = " + texto[0])
# Saida = 'T'

#retornando o último caractere
print("Retorna o último caractere da variavel texto = " + texto[-1])
# Saida = 'l'

print("Retorna os caracteres da variavel texto entre as posições 15 até 21 = " + texto[15:21])
# Saida = 'Python'
```

Listagem 7

FUNÇÕES DA CLASSE **str**

A classe **str** contém um conjunto de função para manipularmos **string**, isto é, cadeias de caracteres. É importante lembrar, que objetos do tipo **str** são **imutáveis** e assim, qualquer alteração que alguma função fizer, a nova **String** que for retornada, será um novo objeto, e este, possuirá um endereço diferente da **String** utilizada inicialmente.

Quantidade de caracteres

A função **built-in len()** pode ser utilizada para retornar a quantidade de caracteres utilizados numa determinada estrutura.

```
texto = "Toda string em Python é imutável"

tamanho = len(texto)

print("Comprimento total da variavel texto = " + str(len(texto)))
print("Resultado da variavel tamanho = " + str(tamanho))
```

Listagem 8

ALTERAÇÃO DE **string**

Como estudado, não é possível alterarmos uma **string**. Assim, se tentarmos atribuir um novo caractere para determinada posição, será retornado um erro, como pode ser visto no código a seguir.

```
texto = "Toda string em Python é imutável"

print(texto[5])

print(texto[7])

texto[5] = "x" #Cuidado!!!
```

Listagem 9

Traceback (most recent call last):

File "g:/IPE-Python2019/Aula06-String_Python/exemplo63.py", line 30, in <module>

texto[5] = "x" #Cuidado!!!

TypeError: 'str' object does not support item assignment

Inicialmente, utilizamos a notação para que fosse retornado o caractere de índice igual a 4 e 7. Em seguida, tentamos alterar o caractere de índice igual a 5 e uma exceção foi gerada.

Função split()

A função **split()** divide um texto todas as vezes que a **String** passada como argumento for localizada. A listagem 10, definimos uma frase e sem seguida, invocamos a função **split()**, definindo como argumento, uma **String** que contém um único espaço em branco.

```
texto = "Toda string em Python é imutável"
```

```
novaLista = texto.split(" ")
```

```
print(texto.split(" "))
```

```
print(novaLista)
```

```
print(novaLista[3] + " " + novaLista[5])
```

```
print(id(texto))
```

```
print(id(novaLista))
```

Listagem 10

```
['Toda', 'string', 'em', 'Python', 'é', 'imutável']
```

Como podemos ver, uma lista foi retornada e está contém as palavras como sendo elementos. Se observarmos a **String** original, veremos que as quebras foram feitas a cada espaço em **branco existente**.

A função **id()** retorna a identificação do objeto string, cada string tem um **valor único e diferente**

Função replace()

A função **replace()** substitui uma parte do texto por uma outra **String**. A palavra **replace()**, do Inglês, significa substituir e é isso que a função **replace()** da classe String do Python faz.

No código a seguir, iremos definir uma String e em seguida, substituiremos uma parte do texto por outra.

```
s = "A função replace aa substitui parte de um texto por outro texto"
```

```
s2= s.replace("aa", "123")
```

```
print("Frase original = " +s)
```

```
print("Frase alterada = " + s2)
```

```
# 'A função replace 123 substitui parte de um texto por outro texto'
```

```
print(id(s))
```

```
print(id(s2))
```

Listagem 11

Na listagem 11 foi definido uma **String (s)** e dentro desta, colocamos uma marcação **aa**, somente para facilitar o estudo. Em seguida, invocamos a função **replace()**, e dissemos, que queríamos alterar a string **aa** pelo texto, "123". Feito isso, uma nova String foi retornada, com a alteração feita.

Novamente, é importante lembrar, que foi retornado uma **nova String**, até porque, a String utilizada não pode ser alterada.

Relação entre 2 caracteres

Para entendermos como o Python estabelece a relação entre 2 caracteres, precisamos entender alguns conceitos sobre o princípio da computação.

A listagem 12, temos um código onde as expressões serão o resultado da relação entre 2 caracteres.

```
print("O Resultado da comparação entre (A == A) = " + str("A" == "A")) # True

print("O Resultado da comparação entre (b == b) = " + str("b" == "b")) # True

print("O Resultado da comparação entre (a > c) = " + str("a" > "c")) # False

print("O Resultado da comparação entre (C > a) = " + str("C" > "a")) # False
```

Listagem 12

TABELA ASCII

A **tabela ASCII** contém os caracteres vinculados a um número inteiro positivo. Essa tabela decorre do fato de que o computador só manipula números, logo, cada caractere, possui, um número que o identifique.

O termo ASCII é um acrônimo de [American Standard Code for Information Interchange](#) que numa tradução livre, seria, "**Código Padrão Americano para o Intercâmbio de Informação**".

Podemos obter o **código ASCII** de um caractere ou então, através de um caractere, obter o seu respectivo código. Geralmente, trabalhamos com os códigos ASCII quando estamos definindo funcionalidades para determinadas teclas, como por exemplo, na construção de um jogo.

A **tabela ASCII** é a convenção que define o valor decimal e binário de cada caractere. Nesse momento, não aprofundaremos o estudo sobre a **tabela ASCII** ou então, o padrão **UNICODE**. Agora, o que nós temos que saber, é que cada letra do nosso teclado está associado a um valor numérico.

[Veja neste link uma explicação da relação do Python com o sistema UNICODE](#).

Por exemplo, a **letra A maiúscula** recebe o **código 65**, a **letra B** o **código 66** e assim por diante. A **letra a minúscula**, recebe o **código 97**, e a letra **b minúscula** o **código 98** e assim por diante.

Esses valores serão utilizados, por exemplo, quando comparamos 2 String distintas. Por exemplo, se digitar a seguinte expressão, qual o valor lógico retornado?

```
print("a" > "X") # True
```

Listagem 13

Temos que o valor lógico retornado será True, até porque, o código ASCII do caractere a minúsculo (97) é maior do que o caractere X maiúsculo (88).

O Python disponibiliza **2 funções** que são bastante úteis quando estamos trabalhando com o sistema **ASCII**.

Função	Descrição
ord()	recebe uma letra como parâmetro e retorna o código ASCII da mesma.
chr()	passa o código ASCII e nos é retornado a respectiva letra.

A listagem 14 mostra 2 exemplos utilizando as funções builtin do Python `ord()` e `chr()`.

```
print(ord('a')) # 97

print(chr(97)) # 'a'
```

Listagem 14

No exemplo acima, inicialmente, confirmamos que o **código ASCII** da letra *a* minúscula é 97.

Em seguida, passamos o **código 97** como argumento da **função `chr()`** e, como resposta, tivemos o caractere *a*.

A Tabela **ASCII (American Standard Code for Information Interchange)** não é nenhuma novidade para quem já trabalha com informática, principalmente com desenvolvimento de software. A minha intenção em publicar esta tabela é adicionar várias informações em uma única tabela, já que a maioria disponível na internet apresenta poucos dados.

Esta tabela é a junção da

Tabela	Numero
ASCII Normal	32 a 127
Caracteres de Controle	0 a 31
ASCII Estendida	128 a 255

ASCII control characters			ASCII printable characters			Extended ASCII characters										
00	NULL	(Null character)	32	space	64	@	96	`	128	Ç	160	á	192	Ł	224	Ó
01	SOH	(Start of Header)	33	!	65	A	97	a	129	ù	161	í	193	ł	225	ô
02	STX	(Start of Text)	34	"	66	B	98	b	130	é	162	ó	194	Ł	226	Ô
03	ETX	(End of Text)	35	#	67	C	99	c	131	â	163	û	195	ł	227	Õ
04	EOT	(End of Trans.)	36	\$	68	D	100	d	132	ä	164	ñ	196	—	228	ö
05	ENQ	(Enquiry)	37	%	69	E	101	e	133	å	165	Ñ	197	†	229	ő
06	ACK	(Acknowledgement)	38	&	70	F	102	f	134	Å	166	ª	198	ā	230	µ
07	BEL	(Bell)	39	'	71	G	103	g	135	ç	167	º	199	Ä	231	þ
08	BS	(Backspace)	40	(	72	H	104	h	136	ê	168	¿	200	Ē	232	þ
09	HT	(Horizontal Tab)	41	)	73	I	105	i	137	ë	169	®	201	ƒ	233	Ů
10	LF	(Line feed)	42	*	74	J	106	j	138	è	170	™	202	ƒ	234	Ů
11	VT	(Vertical Tab)	43	+	75	K	107	k	139	ĩ	171	½	203	ƒ	235	Ů
12	FF	(Form feed)	44	,	76	L	108	l	140	î	172	¾	204	ƒ	236	ý
13	CR	(Carriage return)	45	-	77	M	109	m	141	ï	173	¿	205	=	237	Ÿ
14	SO	(Shift Out)	46	.	78	N	110	n	142	Ā	174	«	206	ƒ	238	—
15	SI	(Shift In)	47	/	79	O	111	o	143	Ă	175	»	207	ƒ	239	—
16	DLE	(Data link escape)	48	0	80	P	112	p	144	É	176	»	208	ð	240	=
17	DC1	(Device control 1)	49	1	81	Q	113	q	145	æ	177	»	209	Đ	241	±
18	DC2	(Device control 2)	50	2	82	R	114	r	146	Æ	178	»	210	É	242	—
19	DC3	(Device control 3)	51	3	83	S	115	s	147	ø	179	»	211	Ê	243	¼
20	DC4	(Device control 4)	52	4	84	T	116	t	148	ö	180	»	212	Ë	244	¶
21	NAK	(Negative acknowl.)	53	5	85	U	117	u	149	ò	181	À	213	ì	245	§
22	SYN	(Synchronous idle)	54	6	86	V	118	v	150	ú	182	Á	214	í	246	±
23	ETB	(End of trans. block)	55	7	87	W	119	w	151	û	183	Â	215	î	247	·
24	CAN	(Cancel)	56	8	88	X	120	x	152	ÿ	184	©	216	ï	248	°
25	EM	(End of medium)	57	9	89	Y	121	y	153	Û	185	»	217	Ĵ	249	—
26	SUB	(Substitute)	58	:	90	Z	122	z	154	Ü	186	»	218	Œ	250	·
27	ESC	(Escape)	59	;	91	[	123	{	155	ø	187	»	219	Œ	251	·
28	FS	(File separator)	60	<	92	\	124	{	156	£	188	»	220	Œ	252	·
29	GS	(Group separator)	61	=	93	]	125	}	157	ø	189	»	221	Œ	253	·
30	RS	(Record separator)	62	>	94	^	126	~	158	×	190	»	222	Œ	254	·
31	US	(Unit separator)	63	?	95	_			159	f	191	»	223	Œ	255	nbsp
127	DEL	(Delete)														

Figura 2:Tabela ASCII

EXEMPLO - Imprimindo a tabela ASCII

A listagem 14, é um exemplo em que é impresso, na saída padrão, o código ASCII e a respectiva letra dos 122 primeiros caracteres.

```
print("O Resultada da comparação entre (A == A) = " + str("A" == "A")) # True

print("O Resultada da comparação entre (b == b) = " + str("b" == "b")) # True

print ("O Resultada da comparação entre (a > c) = " + str("a" > "c")) # False

print ("O Resultada da comparação entre (C > a) = " + str("C" > "a")) # False

print (ord('a')) # 97

print("Valor do decimal 0 = ", chr(0)) # 'a'

print("Valor do decimal 50 = ", chr(50))

for c in range(127):
    print(str(c) + " == " + chr(c))
```

Listagem 15

```
'0 - \x00'
'1 - \x01'
'2 - \x02'
'3 - \x03'
'4 - \x04'
'5 - \x05'
'6 - \x06'
'7 - \x07'
'8 - \x08'
'9 - \t'
'10 - \n'
'11 - \x0b'
'12 - \x0c'
'13 - \r'
'14 - \x0e'
'15 - \x0f'
'16 - \x10'
'17 - \x11'
'18 - \x12'
'19 - \x13'
'20 - \x14'
'21 - \x15'
'22 - \x16'
'23 - \x17'
'24 - \x18'
'25 - \x19'
'26 - \x1a'
'27 - \x1b'
'28 - \x1c'
'29 - \x1d'
'30 - \x1e'
'31 - \x1f'
'32 - '
'33 - !'
'34 - '"
```

```
'35 - #'
'36 - $'
'37 - %'
'38 - &'
'39 - '"
'40 - ('
'41 - )'
'42 - *'
'43 - +'
'44 - ,'
'45 - -'
'46 - .'
'47 - /'
'48 - 0'
'49 - 1'
'50 - 2'
'51 - 3'
'52 - 4'
'53 - 5'
'54 - 6'
'55 - 7'
'56 - 8'
'57 - 9'
'58 - :.'
'59 - ;'
'60 - <'
'61 - ='
'62 - >'
'63 - ?'
'64 - @'
'65 - A'
'66 - B'
'67 - C'
'68 - D'
'69 - E'
'70 - F'
'71 - G'
'72 - H'
'73 - I'
'74 - J'
'75 - K'
'76 - L'
'77 - M'
'78 - N'
'79 - O'
'80 - P'
'81 - Q'
'82 - R'
'83 - S'
'84 - T'
'85 - U'
'86 - V'
'87 - W'
'88 - X'
'89 - Y'
'90 - Z'
'91 - ['
'92 - \\'
'93 - ]'
'94 - ^'
```

```
'95 - '  
'96 - '\n'  
'97 - a'  
'98 - b'  
'99 - c'  
'100 - d'  
'101 - e'  
'102 - f'  
'103 - g'  
'104 - h'  
'105 - i'  
'106 - j'  
'107 - k'  
'108 - l'  
'109 - m'  
'110 - n'  
'111 - o'  
'112 - p'  
'113 - q'  
'114 - r'  
'115 - s'  
'116 - t'  
'117 - u'  
'118 - v'  
'119 - w'  
'120 - x'  
'121 - y'  
'122 - z'
```

Listagem 14

Caracteres de escape

Os caracteres de escape é uma definição para adicionar as **strings**, marcações que não possuem um desenho (representação visual) propriamente dito.

Ainda que estes caracteres não tenham uma simbologia que os justifiquem, os mesmos possuem representações na forma em que nossos textos serão exibidos ou impressos.

Os Caracteres de Escape desempenham tarefas bastante importantes na formatação e exibição dos textos.

Tomemos por exemplo, uma quebra de linha, ou seja, o pressionamento a tecla **Enter**.

A quebra de linha não possui um caractere que a represente visualmente, até porque, a sua representação visual, é o posicionamento do cursor no canto esquerdo da linha seguinte.

O mesmo acontece com a tabulação, onde não há um desenho que a represente, até porque, o objetivo, é o posicionamento do texto mais a direita do vídeo.

É interessante notar, que os Caracteres de Escape, fazem um trabalho semelhante, ao encontrado em linguagem de marcação moderna, como por exemplo, o HTML, que utiliza para a representação de quebra de linha, a Tag `<br>`.

Notação de escape

A notação utilizada e o que define os Caracteres de Escape, é a barra invertida. Todo Caractere de Escape é formado por uma barra invertida e outro caractere que definirá a ação.

Caractere	Descrição
/n	quebra de linha
/t	Tabulação

Possíveis erros

Alguns Caracteres de Escape, talvez, não funcionem no editor que estejas utilizando, e também, não tente utilizar os Caracteres de Escape dentro de editores de texto puro, como por exemplo, o Bloco de Notas do Windows, até porque, ao invés de obteres um o resultado aqui descrito, terás somente, os dígitos em sua representação textual.

Os Caracteres de Escape, como por exemplo, o Carriage Return, cuja notação é `\r`, normalmente, não funciona na plataforma Windows e nem nos Prompts de Comando modernos. O mesmo foi amplamente utilizado, na época das impressoras matriciais, até porque, praticamente todo o controle era feito com uso de Caracteres de Escape, e o funcionamento da manipulação das impressoras, ocorria de forma semelhante a maneira como manipulamos o prompt atualmente.

A diferença, é que ao invés de enviarmos o texto para o Prompt, enviávamos para um arquivo virtual, que era na verdade, a saída padrão da impressora.

A tabela de caracteres de escape

A seguir, temos as principais Escape Sequence da linguagem C. Os dois primeiro itens, que nos caso, são a quebra de linha `n` e a tabulação `t` são os mais utilizados.

Porém, é importante observar que muitas vezes precisaremos, por exemplo, imprimir a barra invertida, e então, precisaremos "escapar" a mesma, isto é, precisaremos utilizar um caractere especial para inserir num texto que a barra deve ser utilizada visualmente, e não como um Caractere de Escape.

Caractere	Descrição
<i>n</i>	Nova página (line feed)
<i>t</i>	Tabulação (tab)
<i>'</i>	Aspa simples
<i>"</i>	Aspa dupla
<i>0</i>	Caractere null (fim de uma String)
<i>?</i>	Interrogação
<i> </i>	backslash
<i>a</i>	Emissão de som
<i>b</i>	backspace
<i>f</i>	Nova página (form feed)
<i>r</i>	Retorno do cursor (carriage return)
<i>v</i>	Tabulação Vertical

É comum, por exemplo, precisarmos imprimir um texto e em seguida, posicionar o cursor na linha seguinte. Para isso, basta adicionarmos ao término da string um `\n`, que no caso, é a notação que representa uma quebra de linha.

25 métodos para trabalhar com strings

01. capitalize()

Capitalizar uma palavra

```
name = "tTERMINAL"
```

```
print( name.capitalize() )
```

Saída: Terminal

02. casefold()

Converte para minúsculas

```
name = "tTERMINAL"
```

```
print( name.casefold() )
```

Saída: terminal

03. center()

Adicionar caractere no início e no final para totalizar uma quantidade

```
name = "Terminal"
```

```
print( name.center(12, "-") )
```

Saída: --Terminal--

04. count()

Retorna a quantidade de vezes que um caractere se repete

```
name = "casa"
```

```
print( name.count("a") )
```

Saída: 2

05. endswith()

Retorna True(Verdadeiro) se uma string terminal com uma string indicada

```
name = "Terminal"
```

```
print( name.endswith("nal") )
```

Saída: True

06. find()

Retorna a 1ª posição da letra/string desejada

```
name = "Terminal"
```

```
print( name.find("i") )
```

Saída: 4

07. format()

```
name = "Eu {} {} Python!"
```

```
print( name.format("❤️", "amar") )
```

Saída: Eu ❤️ amar Python!

08. format_map()

```
name = {'a': 'Python', 'b': 'curto'}
```

```
print( 'Eu {b} {a}'.format_map(name) )
```

Saída: Eu curto Python

09. isalnum()

Retorna verdadeiro se a string possuir **somente** letras e/ou números.

```
name = "plan9"
```

```
print( name.isalnum() )
```

Saída: True

10. isalpha()

Retorna verdadeiro se a string possuir **somente** letras.

```
name = "plan9"
```

```
print( name.isalpha() )
```

Saída: False

11. isdecimal()

Retorna verdadeiro se a string for um **número decimal**

```
name = "9.36"
```

```
print( name.isdecimal() )
```

Saída: False

12. isdigit()

Retorna verdadeiro se a string for composta de dígitos

```
name = "936"
```

```
print( name.isdigit() )
```

Saída: True

13. islower()

Verifica se as letras estão em minúsculas

```
name = "somos minúsculos..."
```

```
print( name.islower() )
```

Saída: True

14. isspace()

Verifica se todos os caracteres são espaços em branco

```
name = "      "
```

```
print( name.isspace() )
```

Saída: True

15. isupper()

Verifica se todas as letras estão em maiúsculas

```
name = "TERMINAL"
```

```
print( name.isupper() )
```

Saída: True

16. ljust()

Conta o tamanho da string e adiciona à direita o caractere informado até completar o valor desejado.

```
name = "Terminal"
```

```
print( name.ljust(11, '>') )
```

Saída: Terminal>>> . Agora tente o rjust e obtenha o inverso.

17. lower()

Converte todas as letras em minúsculas

```
name = "TeRmInAl RoOt"
```

```
print( name.lower() )
```

Saída: terminal root

18. upper()

Converte todas as letras em maiúsculas

```
name = "TeRmInAl RoOt"
```

```
print( name.upper() )
```

Saída: TERMINAL ROOT

19. replace()

Substitui uma string

```
name = "Terminal User"
```

```
print( name.replace("User", "Root") )
```

Saída: Terminal Root

20. strip()

Remove espaços em branco da direita e da esquerda.

```
name = "  Terminal Root  "
```

```
print( name.strip() )
```

Saída: Terminal Root

21. title()

Torna todas as palavras capitalizadas

```
name = "tErminAl rOOt pYthon"
```

```
print( name.title() )
```

Saída: Terminal Root Python

22. zfill()

Adiciona zeros à esquerda para completar a quantidade desejada.

```
name = ".936"
```

```
print( name.zfill(6) )
```

Saída: 00.936

23. removesuffix()

Remove parte de uma string

```
name = "Terminal"
```

```
print( name.removesuffix("inal") )
```

Saída: Term

24. swapcase()

Troca ordem de minúsculas para maiúsculas e vice-versa

```
name = "TERMINAL root"
```

```
print( name.swapcase() )
```

Saída: terminal ROOT

25. istitle()

Verifica se todas as iniciais estão capitalizadas(em forma de título)

```
name = "Terminal Root Python"
```

```
print( name.istitle() )
```

Saída: True

Python String Methods

Input	Method	Output
'hello WORLD'	.capitalize()	Hello world
'HELLO WORLD'	.lower()	hello world
'hello world'	.upper()	HELLO WORLD
'Python'	.center(10, '*')	**Python**
'HELLO WORLD'	.count('L')	3
'HELLO WORLD'	.index('O')	4
'HELLO WORLD'	.find('OR')	7
'14/09/2022'	.replace('/', '-')	14-09-2022
'14/09/2022'	.split('/')	['14', '09', '2022']
'abc123'	.isalnum()	True
'12345'	.isnumeric()	True
'hello world'	.islower()	True
'HELLO WORLD'	.isupper()	True

Figura 3:Alguns exemplos de métodos strings

Arquivos em Python

A maioria dos programas que vimos até agora são transitórios, porque são executados por algum tempo e produzem alguma saída, mas, quando terminam, seus dados desaparecem. Se executar o programa novamente, ele começa novamente do zero.

Outros programas são persistentes:

- a) **rodam por muito tempo (ou todo o tempo);**
- b) **mantêm pelo menos alguns dos seus dados em armazenamento permanente (uma unidade de disco rígido, por exemplo); e**
- c) **se são desligados e reiniciados, continuam de onde pararam.**

Exemplos de programas persistentes são sistemas operacionais, que rodam praticamente durante todo o tempo em que um computador está ligado, e servidores web, que rodam todo o tempo, esperando pedidos de entrada na rede.

Para os nossos propósitos, suporemos que existe um arquivos de dados contém texto—isto é, arquivos de caracteres. Os programas em Python que você escreve são armazenados em arquivos de texto. Podemos criar esses arquivos de diversas maneiras.

Por exemplo,

- a) podemos usar um editor de textos para **digitar e salvar** os dados.
- b) Podemos também **baixar (download)** os dados de uma página na **Internet (website)** e salvá-los em um arquivo.

Não importa como os arquivos são criados, Python nos permitirá manipular o seu conteúdo.

Uma alternativa é armazenar o estado do programa em um banco de dados.

Em Python, devemos **abrir (open)** arquivos antes de usá-los e **fechar (close)** os arquivos depois de que tivermos terminado de utilizá-los.

Como você pode imaginar, depois de aberto um arquivo passa a ser um **objeto Python** de maneira semelhante que outros dados.

Encontrando um arquivo disco

Abriu um arquivo requer que você, como programador, e o Python entre em acordo sobre onde está o arquivo no seu disco. A maneira que os arquivos são localizados no disco e através do seu caminho (path). Você pode pensar em nome_arquivo como o primeiro nome do arquivo e o caminho como o nome completo.

Por exemplo em um Mac se o usuário hofstadter salvar o arquivo penny.txt no seu diretório (home directory) o caminho é /Users/hofstadter/penny.txt. No Linux caminho poderia ser /home/hofstadter/penny.txt .

No computadores com o sistema operacional Windows o caminho parece um pouco diferente, mas os princípios são os mesmos. Por exemplo no Windows o caminho poderia ser

F:\Medina Pen2020\IPE-Python2020\Aula06-String_Arquivos-Python\Arquivos

Você também pode acessar arquivos em pastas (folders), também chamados de diretórios (directories), no seu diretório adicionando um barra e o nome da pasta.

Aqui está uma regra importante para ser lembrada: se o seu arquivo de dados e o seu programa Python estão no mesmo diretório você pode usar simplesmente o nome do arquivo. `open('penny.txt','r')` . Se o seu arquivo de dados e o seu programa Python estão em diretórios diferentes então você deve usar o caminho até o arquivo `open('/Users/hofstadter/penny.txt','r')`.

Nomes de arquivo e caminhos

Os arquivos são organizados em diretórios (também chamados de “pastas”). Cada programa em execução tem um “diretório atual”, que é o diretório-padrão da maior parte das operações.

Por exemplo, quando você abre um arquivo de leitura, Python o procura no diretório atual.

O módulo `os` fornece funções para trabalhar com arquivos e diretórios (“os” é a abreviação de “sistema operacional” em inglês).

`os.getcwd` devolve o nome do direto

```
import os

diretorioCorrente = os.getcwd()
print("Vc esta no seguinte diretorio")
print(diretorioCorrente)
```

Listagem 15

Leitura de textos

Um arquivo de texto é uma sequência de caracteres armazenados em um meio permanente como uma unidade de disco rígido, pendrive ou CD-ROM.

Tabela 1 mostra os métodos que podem ser usados para abrir e fechar arquivos.

Nome de Método	Uso	Explicação
open	<code>open(nome_arquivo, 'r')</code>	✓ Abre um arquivo chamado <code>nome_arquivo</code> e o usa para leitura. ✓ Retorna uma referência para um objeto file.
open	<code>open(nome_arquivo, 'w')</code>	✓ Abre um arquivo chamado <code>nome_arquivo</code> e o usa para escrita. ✓ Retorna uma referência para um objeto file.
close	<code>ref_arquivo.close()</code>	✓ Utilização do arquivo referenciado pela variável <code>ref_arquivo</code> terminou. ✓ Quando estamos trabalhando com arquivos, devemos nos preocupar em fechá-lo.

Tabela 1

Modos de abertura

Os principais modos de abertura de arquivos no Python são:

Modo	Descrição
"r"	✓ Abre arquivo somente para leitura. ✓ O ponteiro do arquivo é localizado no início do arquivo.
"r+"	✓ Abre arquivo tanto para leitura quando para escrita. ✓ O ponteiro do arquivo é localizado no início do arquivo.

"w"	✓ Abre arquivo somente para escrita. ✓ Se o arquivo não existe, automaticamente cria o arquivo, se existe, o sobrescreve.
"w+"	✓ Abre arquivo para leitura e escrita. ✓ Assim como os dois acima, automaticamente cria o arquivo, se existe, o sobrescreve.
"a"	✓ Abre arquivo somente para acrescentar (append). ✓ Ponteiro é localizado no fim do arquivo se arquivo já existe. ✓ Caso contrário, automaticamente ele irá cria-lo
"a+"	✓ Abre arquivo tanto para ler quanto para acrescentar. ✓ O ponteiro é localizado no fim do arquivo caso ele já exista. ✓ Caso contrário, automaticamente ele irá cria-lo

Caso não mencionarmos o modo de abertura do arquivo, "r" é utilizado por padrão.

Para arquivos com dados binários

Arquivos podem conter tanto texto quanto dados binários, e para isto há adicionalmente modos de abertura no formato binário, onde geralmente há um sufixo "b" no nome, como em "rb", "rb+", "wb", "wb+", "ab" e "ab+".

Método	Uso	Efeito
read(n)	ref_arquivo.read()	✓ Lê e retorna um string de n caracteres ou o arquivo inteiro como um string se n não é fornecido. ✓ Para abrir um arquivo somente para leitura em um programa Python usamos a função open() tendo como argumento o nome e o caminho até arquivo, o caminho pode ser absoluto ou relativo, e como opcional um segundo argumento o caractere "r". ✓ No terminal abra o interpretador Python dentro do diretório pai do diretório data recém criado e depois tente abrir o arquivo saudacao.txt para leitura conforme mostrado na listagem16, de estiver no Windows lembre-se de usar \ ao invés de /:
readline(n)	ref_arquivo.readline()	✓ Retorna a próxima linha do arquivo com todo o texto e incluindo o caractere de nova linha. ✓ Podemos também ler uma linha por vez com o método readline(): ✓ Se n é fornecido como argumento então somente n caracteres são retornados se a linha tem mais do que n caracteres.
readlines(n)	ref_arquivo.readlines()	✓ Retorna uma lista de strings, cada um representado o conteúdo de uma linha do arquivo. ✓ Uma outra forma de lermos arquivos é com o método readlines() esse método retorna uma lista contendo as linhas do arquivo como itens ✓ Se n não é fornecido, todas as linhas do arquivo são retornadas. ✓ Se n é fornecido como argumento então n caracteres serão lidos mas n é arredondado para cima de tal forma que uma linha inteira seja retornada.

Tabela 3

Lendo arquivos

Podemos ler o conteúdo de um arquivo de diversas maneiras, e a primeira que vamos ver é com o método `read`. Esse método aceita como argumento o número de bytes a serem lidos, e quando esse argumento não é fornecido o método lê o conteúdo todo do arquivo. Exemplo:

```
1 arquivo = open(r"C:/arquivo.txt", 'r')
2 print(arquivo.read())
3 arquivo.close()
```

No exemplo acima lemos e imprimimos todo o conteúdo do arquivo criado no primeiro exemplo. Abaixo fornecemos um número de bytes a serem lidos, exemplo:

```
1 arquivo = file(r"C:/arquivo.txt", 'r')
2 print(arquivo.read(5))          # Imprime Linha
3 arquivo.close()
```

Outra forma de ler arquivos é pelo método `readline`, que lê uma linha inteira do arquivo. Exemplo:

```
1 >>> arquivo = file(r"C:/arquivo.txt", 'r')
2 >>> arquivo.readline()          # Imprime Linha 1
3 >>> arquivo.readline()          # Imprime Linha 2
4 >>> arquivo.close()
```

```
import os

diretorioCorrente = os.getcwd()
print("Vc está no seguinte diretorio")
print(diretorioCorrente)

arq = open('saudacao.txt', 'r', encoding="utf-8")

saudacao = arq.read()
print(arq.read())
#print(arq.readline())
#print(arq.readlines())

lista_saudacao = saudacao.split('\n')
print(lista_saudacao)

arq.close()
```

Listagem 16

No interpretador nos abrimos o arquivo `saudacao.txt` passando o caminho relativo como argumento da função **`open()`**, lemos o seu conteúdo com o método **`read()`** que coloca todo o conteúdo do arquivo em uma string única, depois disso nós fechamos o arquivo uma vez que já temos os dados dele para trabalhar.

Repare que cada frase da string tem no final um caractere de nova linha “`\n`”. Que tal usarmos o método **`split()`** para strings e criarmos uma lista de strings:

No exemplo da listagem 16 olhando para o último elemento da lista podemos ver claramente que o método **`read()`** criou uma nova linha em branco quando leu o arquivo.

No exemplo da listagem 16 abrimos o arquivo para leitura sem passar o argumento ‘`r`’ pois para abrir arquivos no modo somente leitura tanto faz usar o segundo argumento, no caso de abrimos arquivos para escrita o segundo argumento é obrigatório como veremos na próxima seção.

```
import os

diretorioCorrente = os.getcwd()
print("Vc está no seguinte diretorio")
print(diretorioCorrente)

arq = open('arquivo.txt', 'r', encoding='utf-8')
print("Metodo read()")
print(arq.read())

arq.seek(0) #Volta para o inicio do codigo
print("Metodo readline()")
print(arq.readline())
print(arq.readline())

arq.seek(0) #Volta para o inicio do codigo
print("Metodo readlines()")
print(arq.readlines())

arq.close()
```

Listagem 17

```
import os

diretorioCorrente = os.getcwd()
print("Vc está no seguinte diretorio")
print(diretorioCorrente)

arq = open('arquivo.txt', 'r', encoding='utf-8')
print("Metodo read()")
print(arq.read())

arq.seek(0) #Volta para o inicio do codigo
print("Metodo readline()")
print(arq.readline())
print(arq.readline())

arq.seek(0) #Volta para o inicio do codigo
print("Metodo readlines()")
print(arq.readlines())

arq.close()
```

Listagem 18**Métodos `readline()` e `readlines()`**

Além do método `read()`, visto anteriormente, também podemos ler o conteúdo de um arquivo usando os métodos `readline()` e `readlines()`.

Método `readline()`

Retorna uma **linha do texto** a cada chamada, na ordem em que aparecem no arquivo (um ponteiro de linha é incrementado a cada nova chamada ao método).

Método `readlines()`

Retorna uma **lista de valores de string do arquivo**, sendo que cada string corresponde a uma linha do texto.

É mais fácil trabalhar com o método **readlines()** do que com o método **read()**.

Iteradores

Objetos do tipo file suportam iteradores, que serão vistos em Programação Funcional, o que permite iterar sobre o objeto file em um loop for sem precisar chamar nenhuma função explicitamente. Essa solução, além de gerar um código mais simples, é mais rápida e eficiente em termos de memória. Exemplo:

```
1 arquivo = open(r"C:/arquivo.txt", 'r')
2 for linha in arquivo:
3     print linha,
4 arquivo.close()
```

Repare que inserimos uma vírgula no final da instrução print linha, na linha 3, o que faz com que a função print não insira um "\n" após ser executada. Essa alternativa é uma abordagem mais simples, mas que não fornece nenhum tipo de controle mais refinado. O método readlines é ainda outro meio de ler o conteúdo de um arquivo em Python, ele retorna uma lista de strings contendo as linhas do arquivo. Exemplo:

```
1 arquivo = file(r"C:/arquivo.txt", 'r')
2 print arquivo.readlines()      # Imprime Linha
3 arquivo.close()
```

Posicionando-se em arquivos

Para obter a posição corrente em um arquivo utilizamos o **método tell()**, que retorna um inteiro medido em bytes a partir do início do objeto file. Exemplo:

```
1 arq = file(r'C:/arquivo.txt')
2 conteudo_1 = arq.read(5)
3 print arq.tell()              # Imprime 5
```

Para alterar a posição atual em um arquivo utilizamos o **método seek()**, que recebe dois argumentos. O primeiro é o offset – o número de bytes em queremos nos mover – e o segundo é a referência de onde queremos nos localizar. Utilizando como referência o valor 0 (default), nos posicionamos em relação ao início do arquivo; o valor 1 faz com que nos posicionemos em relação à posição atual e 2 utiliza como referência o final do arquivo. Exemplo:

```
1 arq = open(r'C:/arquivo_2.txt', 'w')
2 arq.write('123456789abcd')
3 arq.close()
4
5 arq = open(r'C:/arquivo_2.txt')
6 arq.seek(5)
7 print arq.read(1)            # Imprime 6
8 arq.close()
```

No exemplo acima posicionamos o cursor no sexto byte do arquivo em relação ao seu início. Exemplo utilizando o final do arquivo como referência:

```
1 arq = file(r'C:/arquivo_2.txt')
2 arq.seek(-4, 2)
3 print arq.read(4)            # Imprime abcd
4 arq.close()
```

Como pode perceber nesse exemplo, ao passarmos um offset negativo, recuamos o ponteiro do arquivo, nesse caso, em -4 bytes.

```
#Exemplo 67-a
import os

diretorioCorrente = os.getcwd()
print("Vc está no seguinte diretorio")
print(diretorioCorrente)

arq = open('Aula06-String_Arquivos-Python/Arquivos/arq01.txt', 'r', encoding="utf-8")

saudacao = arq.read()
print("\nConteudo da variavel saudacao\n")
print(saudacao)

lista_arquivo = saudacao.split('\n')
print(lista_arquivo)

print("\n Relatorio do arq com o Método read()\n")
for linha in lista_arquivo:
    print (linha)

arq.close()
```

Listagem do Exemplo 67-a.py: Com read(), iterador for

```
#Exemplo 67-c
import os

diretorioCorrente = os.getcwd()
print("Vc está no seguinte diretorio")
print(diretorioCorrente)

arq = open('Aula06-String_Arquivos-Python/Arquivos/arq01.txt', 'r', encoding="utf-8")

saudacao = arq.readline()
print("\nConteudo da variavel saudacao\n")
print(saudacao)

lista_arquivo = saudacao.split()

print("\n Relatorio do arq com o Método readline()\n")
for linha in lista_arquivo:
    print (linha)

arq.close()
```

Listagem do Exemplo 67-b.py: Com readline(), iterador for

Metodo rstrip()

Podemos eliminar essa linha em branco usando o método rstrip(), # que elimina espaços no início de uma string, pois os caracteres \n são tratados com espaços

```
#Exemplo 67-c
import os

diretorioCorrente = os.getcwd()
print("Vc está no seguinte diretorio")
print(diretorioCorrente)

arq = open('Aula06-String_Arquivos-Python/Arquivos/arq01.txt', 'r', encoding="utf-8")

saudacao = arq.readlines()
print("\nConteudo da variavel saudacao\n")
print(saudacao)

# Podemos eliminar essa linha em branco usando o método rstrip(),
# que elimina espaços no início de uma string, pois os caracteres \n
# são tratados com espaços

print("\n Relatorio do lista_arquivo\n")
for linha in saudacao:
    linha = linha.rstrip()
    print(linha)

arq.close()
```

Listagem do Exemplo 67-c.py: Com readlines(), iterador for o método rstrip()

Atributos e métodos de arquivos

Assim como qualquer objeto em Python, arquivos possuem atributos e métodos relacionados. A seguir veremos alguns dos atributos que objetos do tipo file possuem:

Atributos	Descrição
closed	Retorna True se arquivo está fechado e False, caso não;
mode	Retorna o modo de abertura do arquivo;
name	Retorna o nome do arquivo;

Exemplos:

```
1 arquivo = file(r'C:/arquivo.txt', 'w')
2 print """\
3 Nome do arquivo: %s
4 Modo de abertura: %s
5 Está fechado?: %s"" % (arquivo.name, arquivo.mode, arquivo.closed)
```

Que imprime:

```
Nome do arquivo: C:/arquivo.txt
Modo de abertura: w
Está fechado?: False
```

Fechando um arquivo

Uma boa prática é sempre fechar um arquivo após o término das operações sobre ele. Para fechar um arquivo, usamos o **método close()** no objeto, como em:

arquivo.close()

Os objetos file possuem um atributo **closed** que retorna True se o arquivo estiver fechado, e retorna False se o arquivo ainda estiver aberto. Desta forma, é possível verificar se um arquivo foi realmente fechado ou não. Veja o exemplo a seguir:

```
print('O arquivo está fechado?'.format(arq.closed))
if not arq.closed:
    arq.close()

print('Estado atual Arquivo = {}'.format(arq.closed))
```

Listagem: Verificação do fechamento do arquivo

A opção with

É fundamental e lembrar de fechar o arquivo se não ao sair do interpretador ou do programa em execução iremos perder todas as escritas feitas no arquivo.

Para não correremos esse risco podemos usar o comando with sua sintaxe é:

```
with open(arquivo, 'w') as arq:
    suíte
```

Listagem 19

```
import os

diretorioCorrente = os.getcwd()
print("Vc está no seguinte diretorio")
print(diretorioCorrente)

with open('Aula06-String_Arquivos-Python/Arquivos/arquivo.txt', 'r', encoding='utf-8') as arq:
    #print(arq.read())
    #print(arq.readline())
    print(arq.readlines())

    conteudo = arq.readlines()
    #conteudo = arq.seek(0)
    #conteudo = arq.readlines()

    print("\n Relatorio do arquivo\n")
    for linha in conteudo:
        linha = linha.rstrip()
        print(linha)
```

Listagem 20

o comando with fecha o arquivo automaticamente assim que saímos de sua suite.

Escrevendo em arquivos de textos

Uma das melhores formas de preservar seus dados depois que o programa é encerrado e colocando eles em um arquivo.

Para escrevermos no arquivo precisamos abrir ele como **open()** é obrigatoriamente passar um segundo argumento que pode ser o caractere 'w' ou 'a' sendo que:

Argumento	Descrição
w	✓ cria um novo arquivo para escrita, ✓ se o arquivo já existir ele irá apagar o arquivo existente por isso tome cuidado.
a	✓ abre um arquivo se ele existir e ✓ começa a adicionar novo texto no final do arquivo, se o arquivo não existir ele cria o arquivo.

Método	Uso	Efeito
write	ref_arquivo.write(s)	✓ Adiciona o string s no final do arquivo. ✓ ref_arquivo deve ser uma referência a um arquivo que foi aberto para escrita ("w").

Tabela 3

Para escrever os dados usamos a função **write()** que leva como argumento a string que desejamos gravar no nosso arquivo:

```
import os

diretorioCorrente = os.getcwd()
print("Vc está no seguinte diretorio")
print(diretorioCorrente)

texto = ""
Boa noite.
Vc está preparado para uma imersão em Python?
Python foi criada por Guido van Rossum
a comunidade Python é muito legal
""

arq = open('arquivo2.txt', 'w', encoding='utf-8')
arq.write(texto)
arq.close()
```

Listagem 21

```
import os

diretorioCorrente = os.getcwd()
print("Vc está no seguinte diretorio")
print(diretorioCorrente)

texto = ""
Hoje, é um bom dia para aprender Python
Principais características
```

Vantagens e desvantagens

```

"""

with open('Aula06-String_Arquivos-Python/Arquivos/arquivo3.txt', 'r+', encoding='utf-8') as arqEscrita:
    arqEscrita.write(texto)
    conteudo = arqEscrita.readlines()
    print("\n Relatorio do arquivo\n")

    for linha in conteudo:
        linha = linha.rstrip()
        print(linha)

```

Listagem 22

Glossário

open

Devemos abrir (*open*) um arquivo antes de ler o seu conteúdo: `ref_arquivo = open(nome_arquivo,"r")`. Devemos também abrir um arquivo antes de escrever nele: `ref_arquivo = open(nome_arquivo,"w")`.

close

devemos fechar (*close*) um arquivo depois que acabamos de manipulá-lo: `ref_arquivo.close()`.

read

Método que lê e retorna o conteúdo inteiro de um arquivo em um string. É frequentemente usado em um comando de atribuição de tal forma que a variável seja uma referência para um string com o conteúdo do arquivo: `string_arquivo = ref_arquivo.read()`

readline

Método que lê e retorna o conteúdo da linha corrente de um arquivo como string: `linha_str = ref_arquivo.readline()`. O caractere de nova linha é incluído no final do string.

readlines

Método que lê e retorna o conteúdo de um arquivo como uma lista de strings. Cada linha do arquivo é representado por um string da lista: `lista_de_linhas = ref_arquivo.readlines()`.

Bibliografia

Python, Escreva seus primeiros programas, Felipe Cruz, Casa doCodigo.

Curso Introductorio de Python, Grupy-Sanca, 09 de maio de 2019

Tutorial Python, Release 2.4.2. Guido van Rossum, Fred L. Drake, Jr., editor. Tradução: Python Brasil

Curso Python 3. Juracy Filho e Leonardo Leitão. Cod3r.com.br

https://aprendacompy.readthedocs.io/pt/latest/capitulo_01.html

<https://www.devmedia.com.br/python-trabalhando-com-variaveis/38644>

<https://cadernodelaboratorio.com.br/2017/12/01/variaveis-em-python3/>

<https://cadernoscicomp.com.br/tutorial/introducao-a-programacao-em-python-3/funcoes/>

LINKS EXTERNOS

- [Documentação Oficial das Strings \(en\)](#)
- [Documentação Oficial \(en\)](#)
- [Aprenda Computação com Python \(pt\)](#)
- [Tutorial Point \(en\)](#)
- [Google Education - Python String \(en\)](#)
- [Understand Common Sequence Data Types in Python – String, Tuple, and List](#)
- [Python e o sistema UNICODE Escape Sequence em C Função Builtin ascii\(\) do Python](#)

Parte 2 – Prática (mão na massa)!!!

Para as atividades práticas vamos utilizar o VS Code, que deve estar previamente configurado para executar o python)

Para a criação dos arquivos (com extensão .py) vamos utilizar o editor de texto

Para executar, deve utilizar o interpretador do VS Code, temos 2 opções

- Apertar o Botão direito e selecionar a opção **RUN PYTHON TERMINAL**
- No menu superior, selecionar Debug -> **START WITHOUT DEBUGGING**

Para limpar o console, utilizar o comando clear ou cls

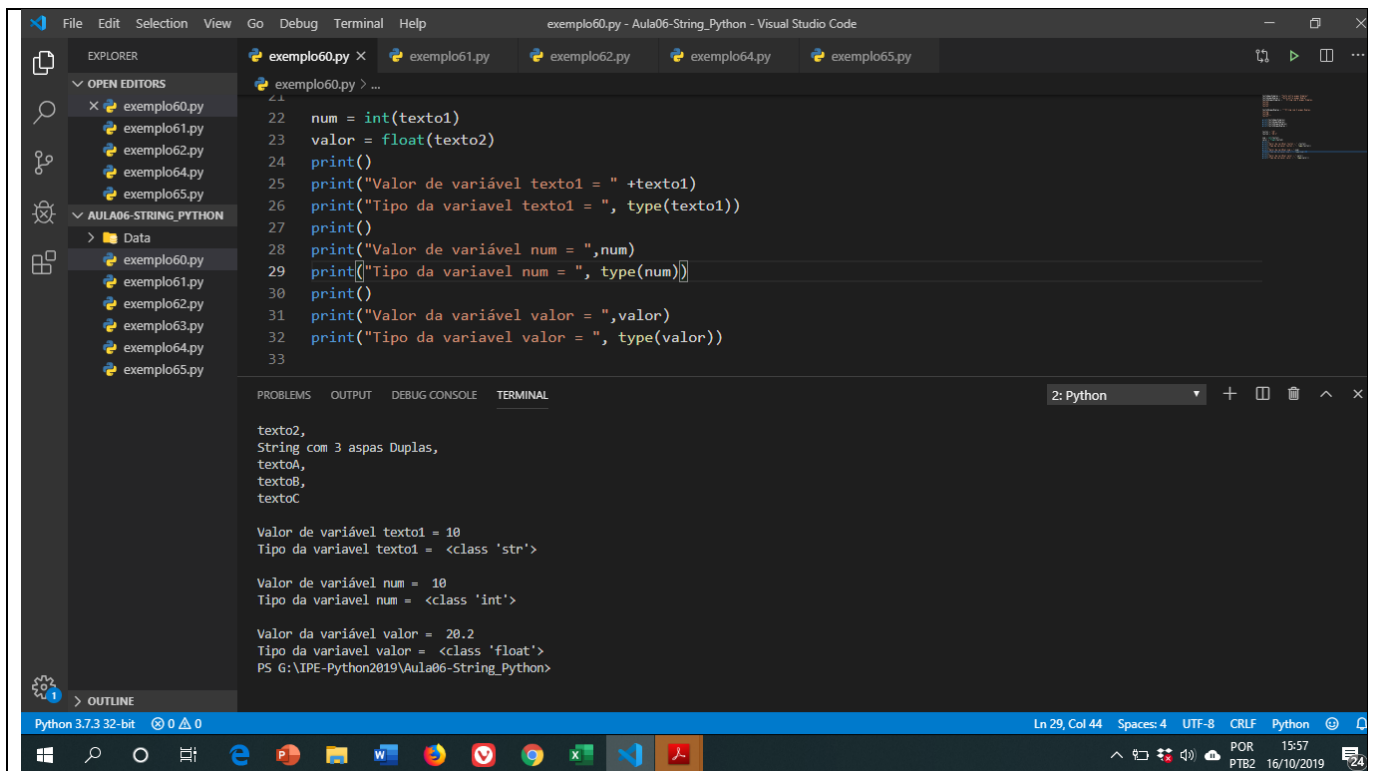
2) Criação do Projeto no VS CODE

- Para isto crie uma pasta no seu pendrive com o nome **IPE-Python-2022**.
- Dentro do **IPE-Python-2022**, criar a pasta **Aula05-Python**
- Utilizando o VS CODE acesse a pasta criada (**IPE-Python-2022/ Aula06-Python**)
- Para isso, acione a opção **File > New > File** e crie os seguintes (exemplo60.py, exemplo61.py, exemplo69.py), para melhor organização (**BOA PRATICA!!!**)

Exemplo 60

PASTA: IPE-Python-2022/ Aula06-Python

Arquivo: exemplo60.py



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left shows a project structure with a folder 'AULA06-STRING_PYTHON' containing a subfolder 'Data' and several Python files (exemplo60.py to exemplo65.py). The main editor window displays the code for 'exemplo60.py'. The code defines variables 'texto1', 'num', and 'valor' and prints their values and types. The output window at the bottom shows the execution results, including the file path 'P5 G:\IPE-Python2019\Aula06-String_Python'.

```
22 num = int(texto1)
23 valor = float(texto2)
24 print()
25 print("Valor de variável texto1 = " + texto1)
26 print("Tipo da variável texto1 = ", type(texto1))
27 print()
28 print("Valor de variável num = ", num)
29 print("Tipo da variável num = ", type(num))
30 print()
31 print("Valor da variável valor = ", valor)
32 print("Tipo da variável valor = ", type(valor))
33
```

terminal

```
texto2,
String com 3 aspas Duplas,
textoA,
textoB,
textoC

Valor de variável texto1 = 10
Tipo da variável texto1 = <class 'str'>

Valor de variável num = 10
Tipo da variável num = <class 'int'>

Valor da variável valor = 20.2
Tipo da variável valor = <class 'float'>
P5 G:\IPE-Python2019\Aula06-String_Python
```

Figura A: exemplo60.py

Exemplo 61

PASTA: IPE-Python-2022/ Aula06-Python

Arquivo: exemplo61.py

```

1  print("Parte I")
2  s = "Lista de Caracteres em Python"
3  print(s)
4
5  print("Parte II") #concatenação de 2 trechos de texto
6  texto1 = "123"
7  texto2 = "456"
8  concatenacao = texto1 + texto2
9  print("Resultado da variavel concatenação = " +concatenacao) # Saida = '123456'
10
11 a = "bra" #declaração da variável `a` que foi associado ao texto `bra`
12 b = "sil" #declaração da variável `b` que foi associado ao texto `sil`
13
14 #ao escrevemos no prompt a + b temos que é impresso a CONCATENAÇÃO
15 print("concatenação de a + b = " + (a + b)) # Saida = 'brasil'
16 c = a + b #agora, concatenamos e o resultado associamos a variável `c`
17 print("Resutado da variavel c = " + c) #Saida = 'brasil'
18 #agora, podemos dizer que o trecho de texto `bra`
19 # foi adicionado a esquerda da variável `b`
20 print("concatenação de b + a = " +(b + a)) #Saida = 'silbra'

```

terminal

```

PS G:\IPE-Python2019\Aula06-String_Python> & C:\Users\SERGIOMEDINA\AppData\Local\Programs\Python\Python37-32\python.exe g:/IPE-Python2019/Aula06-String_Python
/exemplo61.py
Lista de Caracteres em Python
Parte II
Resultado da variavel concatenação = 123456
concatenação de a + b = brasil

```

Figura B: exemplo61.py

Exemplo 62

PASTA: IPE-Python-2022/ Aula06-Python

Arquivo: exemplo62.py

```

1  sexo = "masculino"
2  nome = "Henrique"
3
4  interpolar = "Sexo é igual a %s e o nome é igual %s"%(sexo, nome)
5  print("Saida 1 da interpolação = " +interpolar)
6
7  #agora, vamos colocar uma String entre chaves dentro do texto
8  #em seguida, com a função format(), vamos associar
9  #os nomes definidos entre chaves a texto passando-os
10 #como argumentos de função
11
12 interpolar = "Sexo é igual a {SEXO} e o nome é igual {NOME} ".format(SEXO=sexo,NOME=nome)
13 print("Saida 2 da interpolação = " +interpolar)
14
15 #Saida = 'Sexo é igual a masculino e o nome é igual Henrique '
16

```

terminal

```

Parte II
Resultado da variavel concatenação = 123456
concatenação de a + b = brasil
Resutado da variavel c = brasil
concatenação de b + a = silbra
PS G:\IPE-Python2019\Aula06-String_Python> & C:\Users\SERGIOMEDINA\AppData\Local\Programs\Python\Python37-32\python.exe g:/IPE-Python2019/Aula06-String_Python
/exemplo62.py
Saida 1 da interpolação = Sexo é igual a masculino e o nome é igual Henrique
Saida 2 da interpolação = Sexo é igual a masculino e o nome é igual Henrique
PS G:\IPE-Python2019\Aula06-String_Python>

```

Figura C: exemplo62.py

Exemplo 63

PASTA: IPE-Python-2022/ Aula06-Python

Arquivo: exemplo63.py

```

1  texto = "Toda string em Python é imutável"
2  print("Frase completa = " + texto)
3  print("Retorna o primeiro caractere da variavel texto = " + texto[0]) #retornando o primeiro caractere
4  # Saida = 'T'
5
6  print("Retorna o último caractere da variavel texto = " + texto[-1]) #retornando o último caractere
7  # Saida = 'l'
8
9  print("Retorna os caracteres da variavel texto entre as posições 15 até 21 = " + texto[15:21])
10 # Saida = 'Python'
11
12 print("Retorna os caracteres da variavel texto entre as posições 5 até o final = " + texto[5:])
13 print("Retorna o texto de forma invertida = " + texto[::-1])
14
15 tamanho = len(texto)
16 print("Comprimento total da variavel texto = " + str(len(texto)))
17 print("Resultado da variavel tamanho = " + str(tamanho))
18 print(texto[5])
19 print(texto[7])
20
21 #texto[5] = "x" #Cuidado!!!
22 print(id(texto))

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

r
10392736
PS G:\IPE-Python2019\Aula06-String_Python>

```

Python 3.7.3 32-bit 0 0 0 Ln 15, Col 21 Spaces: 4 UTF-8 CRLF Python 16:03 PTB2 16/10/2019

Figura D: exemplo63.py

Exemplo 64

PASTA: IPE-Python-2022/ Aula06-Python

Arquivo: exemplo64.py

```

1  texto = "Toda string em Python é imutável"
2
3  novalista = texto.split(" ")
4  print(texto.split(" "))
5  print(novalista)
6  print(novalista[3] + " " + novalista[5])
7  teste = novalista[3] + " " + novalista[5]
8  print(id(teste))
9  print(id(texto))
10 print(id(novalista))
11
12 s = "A função replace aa substitui parte de um texto por outro texto"
13 s2= s.replace("aa", "123")
14
15 print("Frase original = " + s)
16 print("Frase alterada = " + s2)
17 # 'A função replace 123 substitui parte de um texto por outro texto'
18
19 print(id(s))
20 print(id(s2))

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

Python imutável
14845392
Frase original = A função replace aa substitui parte de um texto por outro texto
Frase alterada = A função replace 123 substitui parte de um texto por outro texto
14738272
14739312
PS G:\IPE-Python2019\Aula06-String_Python>

```

Python 3.7.3 32-bit 0 0 0 Ln 5, Col 17 Spaces: 4 UTF-8 CRLF Python 16:04 PTB2 16/10/2019

Figura E: exemplo64.py

Exemplo 65

PASTA: IPE-Python-2022/ Aula06-Python

Arquivo: exemplo65.py

```

1 print("0 Resultada da comparação entre (A == A) = " + str("A" == "A")) # True
2 print("0 Resultada da comparação entre (b == b) = " + str("b" == "b")) # True
3 print("0 Resultada da comparação entre (a > c) = " + str("a" > "c")) # False
4 print("0 Resultada da comparação entre (C > a) = " + str("C" > "a")) # False
5
6
7 print(ord('a')) # 97
8
9 print("Valor do decimal 0 = ", chr(0)) # 'a'
10
11 print("Valor do decimal 50 = ", chr(50))
12
13 for c in range(127):
14     print(str(c) + " == " + chr(c))
15

```

```

['Toda', 'string', 'em', 'Python', 'é', 'imutável']
Python imutável
14845392
PS G:\IPE-Python2019\Aula06-String_Python> & C:\Users\SERGIOMEDINA\AppData\Local\Programs\Python\Python37-32\python.exe g:/IPE-Python2019/Aula06-String_Python/exemplo65.py
0 Resultada da comparação entre (A == A) = True
0 Resultada da comparação entre (b == b) = True
0 Resultada da comparação entre (a > c) = False
0 Resultada da comparação entre (C > a) = False
97
Valor do decimal 0 =
Valor do decimal 50 = 2
0 ==
1 ==
2 ==

```

Figura F: exemplo65.py

Arquivos

Exemplo 66

PASTA: IPE-Python-2022/ Aula06-Python

Arquivo: exemplo66.py

```

1 import os
2
3 diretorioCorrente = os.getcwd()
4 print("Vc está no seguinte diretorio")
5 print(diretorioCorrente)
6
7 arq = open('saudacao.txt', 'r', encoding="utf-8")
8
9 saudacao = arq.read()
10 print(arq.read())
11 #print(arq.readline())
12 #print(arq.readlines())
13
14 lista_saudacao = saudacao.split('\n')
15 print(lista_saudacao)
16 arq.close()

```

```

PS G:\IPE-Python2019\Aula06-String_Python\Data> & C:\Users\SERGIOMEDINA\AppData\Local\Programs\Python\Python37-32\python.exe g:/IPE-Python2019/Aula06-String_Python/Data/exemplo66-b.py
Vc está no seguinte diretorio
G:\IPE-Python2019\Aula06-String_Python\Data
PS G:\IPE-Python2019\Aula06-String_Python\Data> & C:\Users\SERGIOMEDINA\AppData\Local\Programs\Python\Python37-32\python.exe g:/IPE-Python2019/Aula06-String_Python/Data/exemplo66.py
Vc está no seguinte diretorio
G:\IPE-Python2019\Aula06-String_Python\Data
['Boa noite ', '', 'Hoje, é um bom dia para aprender Python ', '', 'Vc está pronto? ']
PS G:\IPE-Python2019\Aula06-String_Python\Data>

```

Figura G: exemplo66.py

Exemplo 67-a**PASTA: IPE-Python-2022/ Aula06-Python****Arquivo: exemplo67-a.py**

```

1 import os
2
3 diretorioCorrente = os.getcwd()
4 print("Vc está no seguinte diretorio")
5 print(diretorioCorrente)
6
7 arq = open('arquivo.txt', 'r', encoding='utf-8')
8 print("Metodo read()")
9 print(arq.read())
10
11 arq.seek(0) #Volta para o início do código
12 print("Metodo readline()")
13 print(arq.readline())
14 print(arq.readline())
15
16 arq.seek(0) #Volta para o início do código
17 print("Metodo readlines()")
18 print(arq.readlines())
19 arq.close()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

Vc está no seguinte diretorio
ython/Data/exemplo66.py
Vc está no seguinte diretorio
G:\IPE-Python2019\Aula06-String_Python\Data

['Boa noite ', ' ', 'Hoje, é um bom dia para aprender Python ', ' ', 'Vc está pronto? ']
PS G:\IPE-Python2019\Aula06-String_Python\Data> & C:/Users/SENGIOMEDINA/AppData/Local/Programs/Python/Python37-32/python.exe g:/IPE-Python2019/Aula06-String_Python/Data/exemplo67-a.py

Python 3.7.3 32-bit 0 0 0 Ln 15, Col 1 Spaces: 4 UTF-8 CRLF Python 16:15 16/10/2019

Figura H: exemplo67-a.py**Exemplo 67-b****PASTA: IPE-Python-2022/ Aula06-Python****Arquivo: exemplo67-b.py**

```

1 import os
2
3 diretorioCorrente = os.getcwd()
4 print("Vc está no seguinte diretorio")
5 print(diretorioCorrente)
6
7
8 with open('arquivo.txt', 'r', encoding='utf-8') as arq:
9     print(arq.read())
10     #print(arq.readline())
11     #print(arq.readlines())
12

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS G:\IPE-Python2019\Aula06-String_Python\Data> & C:/Users/SENGIOMEDINA/AppData/Local/Programs/Python/Python37-32/python.exe g:/IPE-Python2019/Aula06-String_Python/Data/exemplo67-b.py
Vc está no seguinte diretorio
G:\IPE-Python2019\Aula06-String_Python\Data
a) 1º HACKATHON UNIP-SANTOS

O primeiro HACKATHON UNIP-SANTOS é um evento voltado aos alunos da Universidade Paulista (UNIP - Santos). Tem como principal objetivo aproximar nossos alunos da vivência de um Hackathon para assim atender as demandas do mercado na área de tecnologia regional. Promovendo possíveis parcerias e fornecer um retorno para a sociedade, através de melhorias em produtos e/ou serviços.

É um evento organizado pelos cursos de Ciência da Computação e Sistemas de Informação. Por se tratar de um evento totalmente voltado aos alunos de graduação no formato será um pouco diferente!

Abaixo algumas informações para você que tem interesse em participar!

PS G:\IPE-Python2019\Aula06-String_Python\Data> []

Python 3.7.3 32-bit 0 0 0 Ln 5, Col 25 Tab Size: 4 UTF-8 CRLF Python 16:16 16/10/2019

Figura I: exemplo67-b.py**Exemplo 68-a**

PASTA: IPE-Python-2022/ Aula06-Python

Arquivo: exemplo68-a.py

The screenshot shows the Visual Studio Code interface with the file explorer on the left, the editor in the center, and the terminal at the bottom. The file explorer shows a project structure with files like arquivo.txt, arquivo2.txt, arquivo3.txt, exemplo66.py, exemplo67-a.py, exemplo67-b.py, exemplo68-a.py, exemplo68-b.py, exemplo69-a.py, exemplo69-b.py, exemploFilmes.py, filmes.txt, and saudacao.txt. The editor shows the code for exemplo68-a.py, which imports os, gets the current directory, prints it, and writes text to arquivo2.txt. The terminal shows the output of the script, which prints the current directory and writes text to arquivo2.txt.

```

1 import os
2
3 diretorioCorrente = os.getcwd()
4 print("Vc está no seguinte diretório")
5 print(diretorioCorrente)
6
7 texto = ''
8 Boa noite.
9 Vc está preparado para uma imersão em Python?
10 Python foi criada por Guido van Rossum
11 a comunidade Python é muito legal
12 '''
13 arq = open('arquivo2.txt', 'w', encoding='utf-8')
14 arq.write(texto)
15 arq.close()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

2: Python

a) 1º HACKATHON UNIP-SANTOS

O primeiro HACKATHON UNIP-SANTOS é um evento voltado aos alunos da Universidade Paulista (UNIP - Santos). Tem como principal objetivo aproximar nossos alunos da vivência de um Hackathon para assim atender as demandas do mercado na área de tecnologia regional.

É um evento organizado pelos cursos de Ciência da Computação e Sistemas de Informação. Por se tratar de um evento totalmente voltado aos alunos de graduação no formato será um pouco diferente!

Abaixo algumas informações para você que tem interesse em participar!

PS G:\IPE-Python2019\Aula06-String_Python\Data>

Figura J: exemplo68-a.py

Exemplo 68-b

PASTA: IPE-Python-2022/ Aula06-Python

Arquivo: exemplo68-b.py

The screenshot shows the Visual Studio Code interface with the file explorer on the left, the editor in the center, and the terminal at the bottom. The file explorer shows a project structure with files like arquivo.txt, arquivo2.txt, arquivo3.txt, exemplo66.py, exemplo67-a.py, exemplo67-b.py, exemplo68-a.py, exemplo68-b.py, exemplo69-a.py, exemplo69-b.py, exemploFilmes.py, filmes.txt, and saudacao.txt. The editor shows the code for exemplo68-b.py, which imports os, gets the current directory, prints it, and reads text from arquivo3.txt. The terminal shows the output of the script, which prints the current directory and reads text from arquivo3.txt.

```

1 import os
2
3 diretorioCorrente = os.getcwd()
4 print("Vc está no seguinte diretório")
5 print(diretorioCorrente)
6
7 texto = ''
8 Python é legal
9 Python foi criada por Guido van Rossum
10 a comunidade Python é muito legal
11 '''
12 arq = open('arquivo3.txt', 'r', encoding='utf-8')
13 texto_arq = arq.readlines()
14 texto_arq.append(texto)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

2: Python

da vivência de um Hackathon para assim atender as demandas do mercado na área de tecnologia regional.

É um evento organizado pelos cursos de Ciência da Computação e Sistemas de Informação. Por se tratar de um evento totalmente voltado aos alunos de graduação no formato será um pouco diferente!

Abaixo algumas informações para você que tem interesse em participar!

PS G:\IPE-Python2019\Aula06-String_Python\Data>

> & C:\Users\SERGIOMEDINA\AppData\Local\Programs\Python\Python37-32\python.exe g:\IPE-Python2019\Aula06-String_Python\Data\exemplo68-b.py

Vc está no seguinte diretório

G:\IPE-Python2019\Aula06-String_Python\Data

PS G:\IPE-Python2019\Aula06-String_Python\Data>

Figura k: exemplo68-b.py

Exemplo 69-a

PASTA: IPE-Python-2022/ Aula06-Python

Arquivo: exemplo69-apy

The screenshot shows the Visual Studio Code interface with the file explorer on the left, the editor in the center, and the terminal at the bottom. The file explorer shows a project structure with files like exemplo66.py, exemplo67-a.py, exemplo67-b.py, exemplo68-a.py, exemplo68-b.py, exemplo69-a.py, arquivo.txt, arquivo2.txt, arquivo3.txt, exemploFilmes.py, filmes.txt, and saudacao.txt. The editor shows the code for exemplo69-apy.py, which imports os, gets the current directory, prints it, opens a file named 'filmes.txt' in write mode, writes two lines of text, and closes the file. The terminal shows the command to run the script and the output, which is the current directory path.

```

1 import os
2
3 diretorioCorrente = os.getcwd()
4 print("Vc está no seguinte diretorio")
5 print(diretorioCorrente)
6
7 arq = open('filmes.txt', 'w', encoding='utf-8')
8 arq.write('Tubarão\n')
9 arq.write('O Poderoso Chefão\n')
10
11 arq.close()
12

```

```

PS G:\IPE-Python2019\Aula06-String_Python\Data> & C:/Users/SERGIONEDINA/AppData/Local/Programs/Python/Python37-32/python.exe g:/IPE-Python2019/Aula06-String_Python/Data/exemplo69-a.py
Vc está no seguinte diretorio
G:\IPE-Python2019\Aula06-String_Python\Data
PS G:\IPE-Python2019\Aula06-String_Python\Data>

```

Figura m: exemplo69-a.py

Exemplo 69-b

PASTA: IPE-Python-2022/ Aula06-Python

Arquivo: exemplo69-bpy

The screenshot shows the Visual Studio Code interface with the file explorer on the left, the editor in the center, and the terminal at the bottom. The file explorer shows the same project structure as the previous screenshot. The editor shows the code for exemplo69-b.py, which imports os, gets the current directory, prints it, opens a file named 'filmes.txt' in append mode, and writes two lines of text. The terminal shows the command to run the script and the output, which is the current directory path.

```

1 import os
2
3 diretorioCorrente = os.getcwd()
4 print("Vc está no seguinte diretorio")
5 print(diretorioCorrente)
6
7 with open('filmes.txt', 'a', encoding='utf-8') as arq:
8     arq.write('\nAliens, O Resgate\n')
9     arq.write('De Volta Para O Futuro 1\n')
10

```

```

PS G:\IPE-Python2019\Aula06-String_Python\Data> & C:/Users/SERGIONEDINA/AppData/Local/Programs/Python/Python37-32/python.exe g:/IPE-Python2019/Aula06-String_Python/Data/exemplo69-b.py
Vc está no seguinte diretorio
G:\IPE-Python2019\Aula06-String_Python\Data
PS G:\IPE-Python2019\Aula06-String_Python\Data> & C:/Users/SERGIONEDINA/AppData/Local/Programs/Python/Python37-32/python.exe g:/IPE-Python2019/Aula06-String_Python/Data/exemplo69-b.py
Vc está no seguinte diretorio
G:\IPE-Python2019\Aula06-String_Python\Data
PS G:\IPE-Python2019\Aula06-String_Python\Data>

```

Figura m: exemplo69-b.py